Министерство науки и высшего образования Российской Федерации

Федеральное государственное бюджетное образовательное учреждение высшего образования
«Томский государственный университет систем управления и радиоэлектроники»

Факультет систем управления (ФСУ)

Кафедра автоматизированных систем управления (АСУ)

Создание документированного веб сервиса на flask. Swagger.

Отчет по лабораторной работе №4 по дисциплине
«Разработка веб-сервисов для научных и прикладных задач»

Обучающийся гр. 431-3
_____ Андреев Д.П.
«___» _____ 2022 г.

Проверил: Кандидат технических наук
_____ Суханов А.Я.
«___» _____ 2022 г.

Томск 2022

**Оглавление**

1. Введение ..... 3
2. Теория ..... 4
3. Ход выполнения работы ..... 4
3.1. Создание тестового веб-приложения ..... 4
3.2. Создание веб-приложение по варианту и индивидуальное задание ..... 5
4. Вывод ..... 12
5. Приложение ..... 13

# 1. Введение

**Цель** - Изучение создания веб-сервиса с использованием фреймворка Flask и его документирование с использование Swagger.

**Задания:**

Необходимо повторить описанные в главе 3 примеры и создать свой собственный веб-сервис и API, реализующий возврат данных по заданию в соответствии с вариантом, где задана простая предметная область. Необходимо разработать простую модель данных, в соответствии с которой будут храниться данные, соответствующие предметной области. Количество полей должно быть более четырех, два-три поля могут быть строкового или другого типа, остальные – числового.

Веб-сервис должен предоставлять возможность сортировки по всем полям записей, выдавать среднее, максимальное и минимальное значение по числовым полям, добавлять, удалять записи и обновлять записи, например, по идентификатору.

Для самостоятельного задания был выдан вариант 1. Этот вариант нацелен на создания базы данных по студентам, изучающих дисциплины в университете.

## 2. Теория

В методичке было рассказано про то, что есть множество ресурсов, представляющих возможности для создания веб-приложений с возможностью непрерывной интеграции с github-ом.

Создание веб-приложения было решено с помощью операционной системы Arch linux и библиотеки Flask для создания веб-приложения. Для создания дизайна и интерфейса потребуется знание html, css и шаблоны Flask.

## 3. Ход выполнения работы

К сожалению все американские сервисы для создания непрерывной интеграции веб-сервисов просили номера карточки. Поэтому было решено выполнять задание на локальной сети. Все коды в приложении.

### 3.1. Создание тестового веб-приложения

В методичке было проведено много примеров для работы с Flask. Результаты некоторых (Код в приложении) вы сможете увидеть внизу в виде картинок (Рисунки 3.1-3.2)

```
[jenya@KnockKnockPC WEBLAB4]$ python main.py
 * Serving Flask app 'main' (lazy loading)
 * Environment: production
   WARNING: This is a development server. Do not use it in a production deployme
nt.
   Use a production WSGI server instead.
 * Debug mode: on
 * Running on http://127.0.0.1:5000 (Press CTRL+C to quit)
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 686-483-348
127.0.0.1 - - [13/Jun/2022 18:05:23] "GET / HTTP/1.1" 404 -
```

Рисунок 3.1 – запуск тестового веб-сервиса из методички в локальной сети

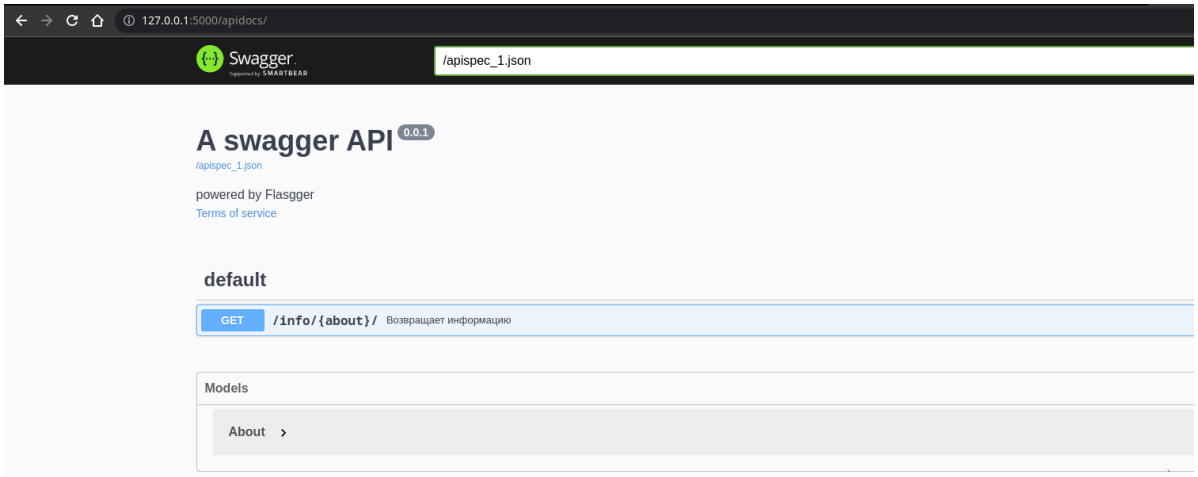


Рисунок 3.2 – тестовое веб-приложение с выводом информации от Swagger.

### 3.2. Создание веб-приложение по варианту и индивидуальное задание

Задание заключается в создании веб-приложения на Flask. Для работы был дан вариант 1.

Приложение должно возвращать необходимые данные о студентах, изучающих дисциплины в университете. Нужно разработать модель базы данных, количество полей которой должно быть больше 4-х (id, фио студента, любимый предмет, группа студента, средний балл студента). Нужно создать также возможность сортировки по всем полям записей, выдавать среднее, максимальное и минимальное значение по числовым полям. Также нужно добавить возможность удалять, изменять и добавлять новые записи.

В качестве сервера было решено использовать локальный компьютер. Внизу представлены рисунки результатов работы web-приложения. (Рисунки 3.3-3.14)

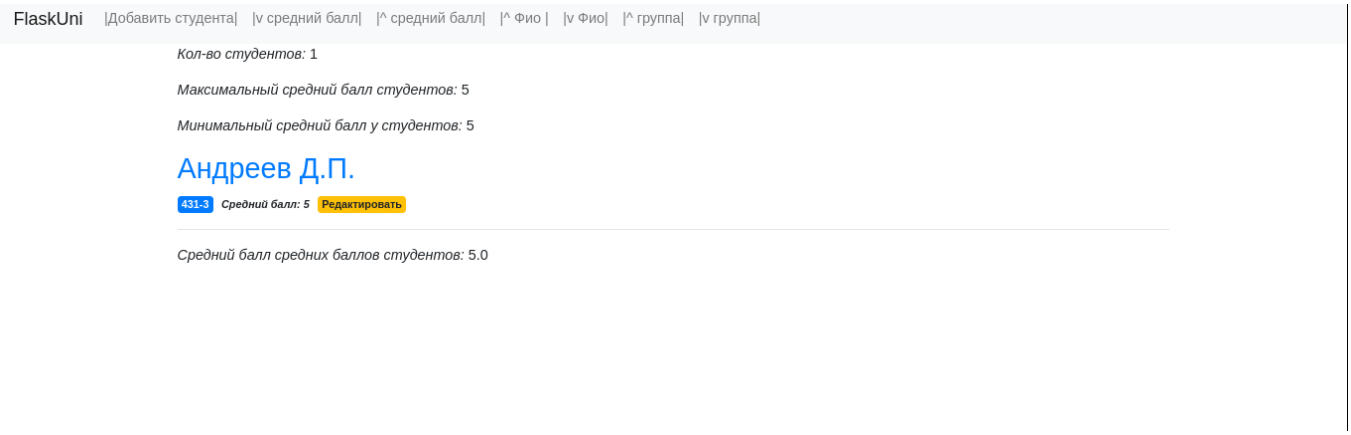


Рисунок 3.3 – главная страница веб-приложения

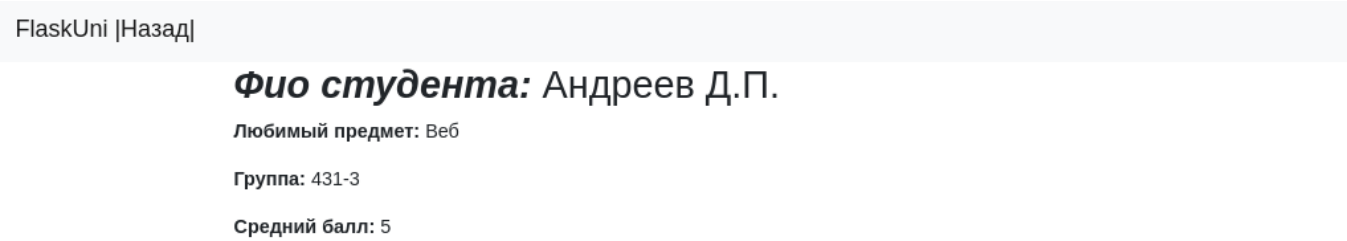


Рисунок 3.4 – страница чтения странички студента

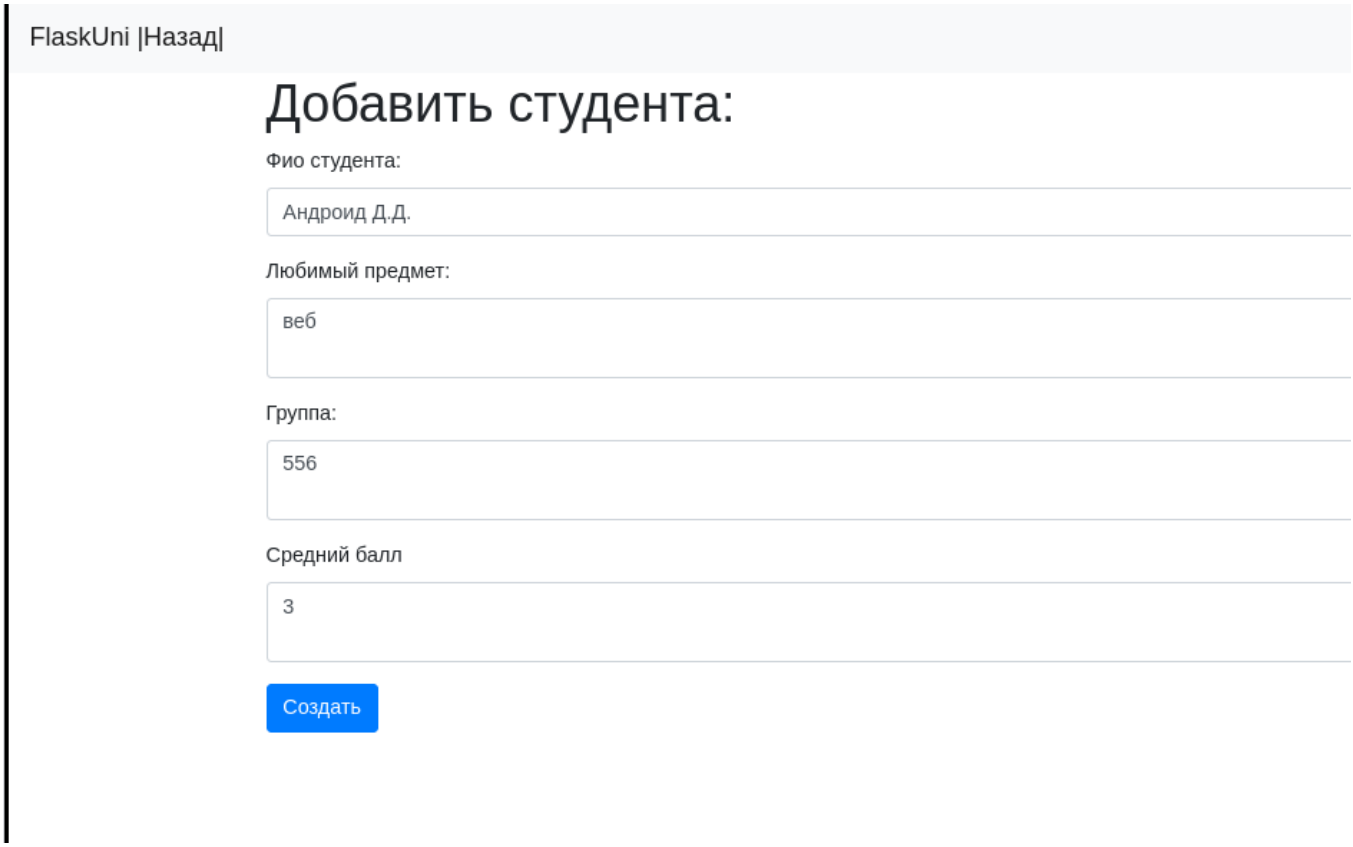


Рисунок 3.5 – страница добавления студента

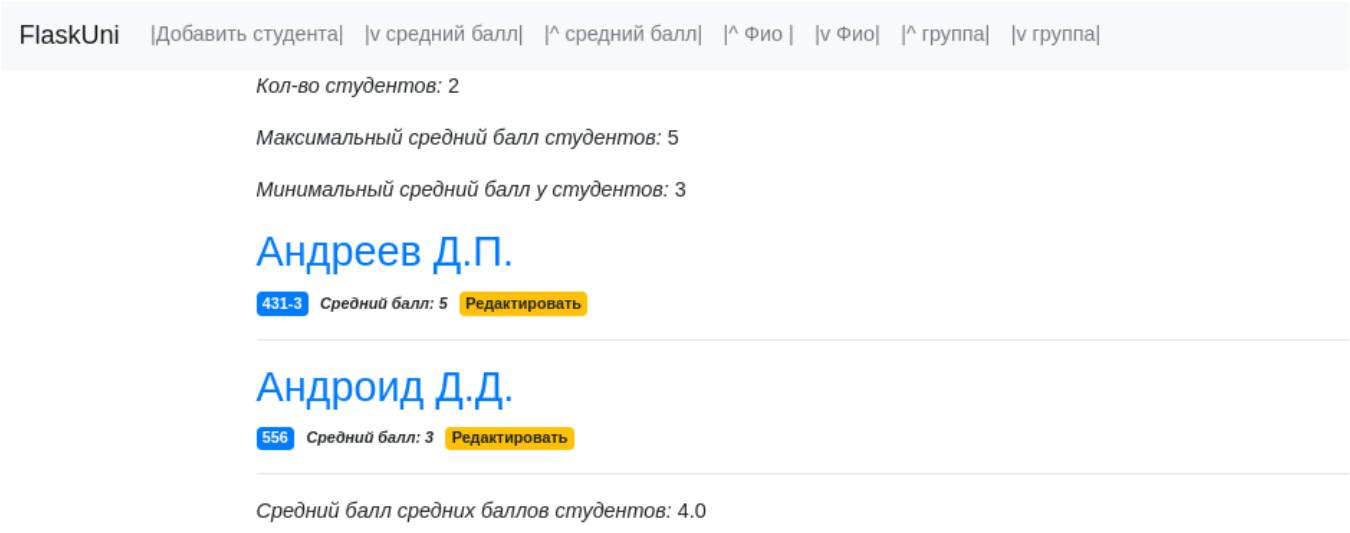


Рисунок 3.6 – вывод результата работы создания нового элемента, а также выявление максимальной, минимальной и среднего среднего балла студентов

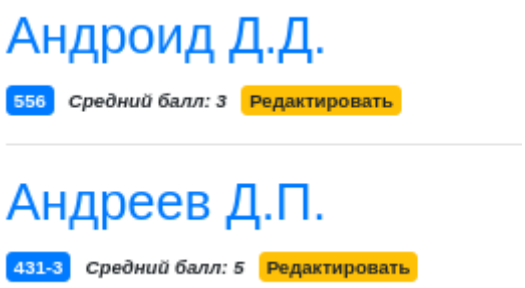


Рисунок 3.7 – результат сортировки среднего балла по возрастанию

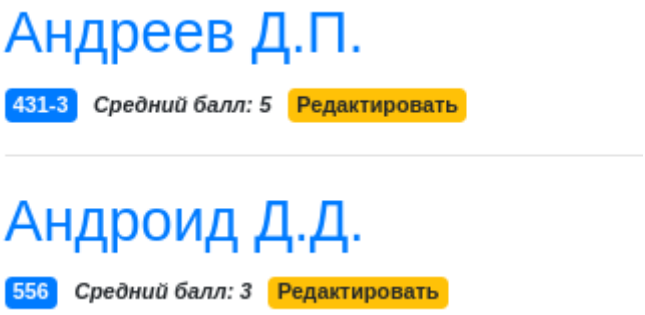


Рисунок 3.8 – результат сортировки цены по убыванию

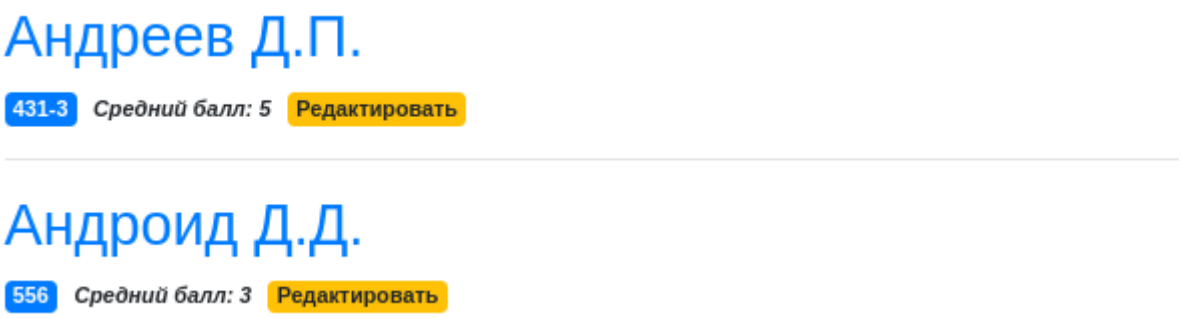


Рисунок 3.9 – результат сортировки по фамилии студента

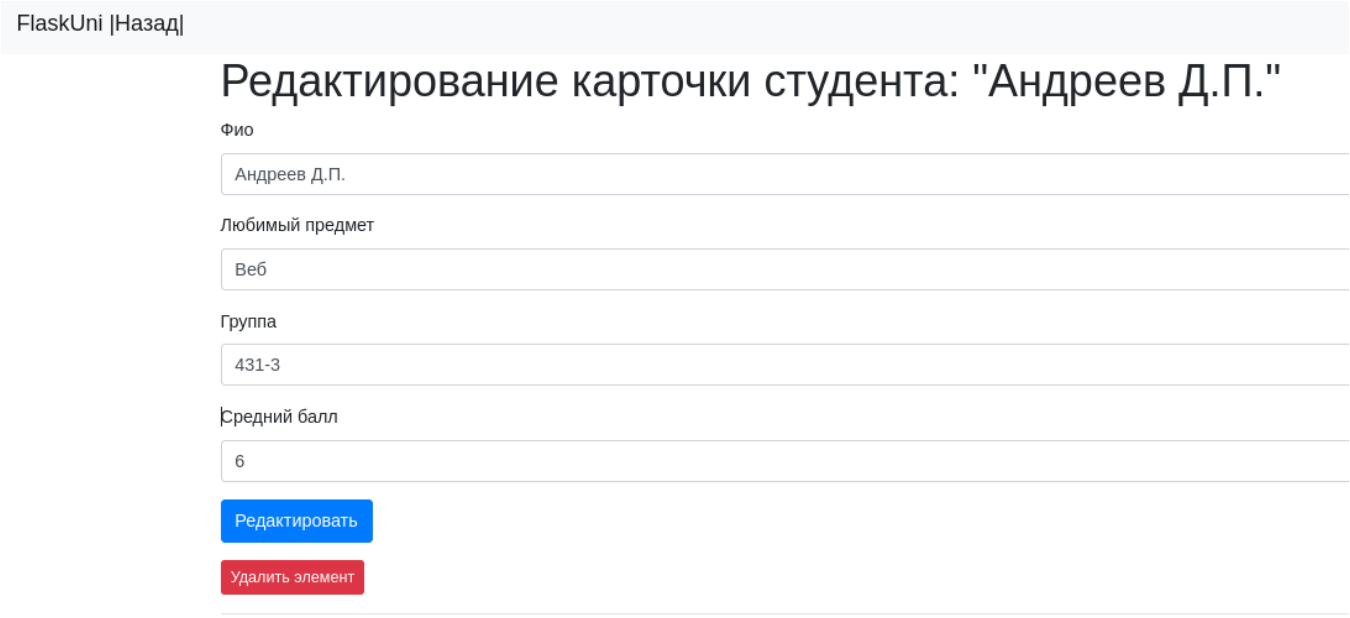

Рисунок 3.10 – страница редактирования элемента

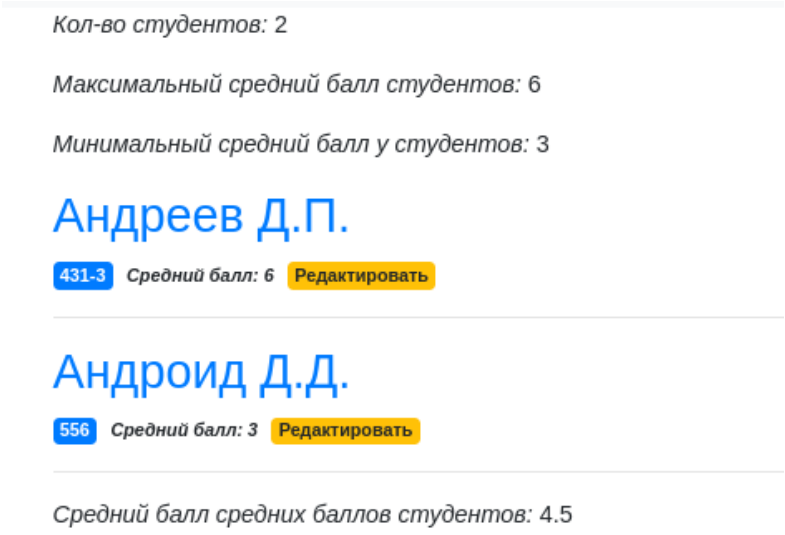


Рисунок 3.11 – результат редактирования элемента

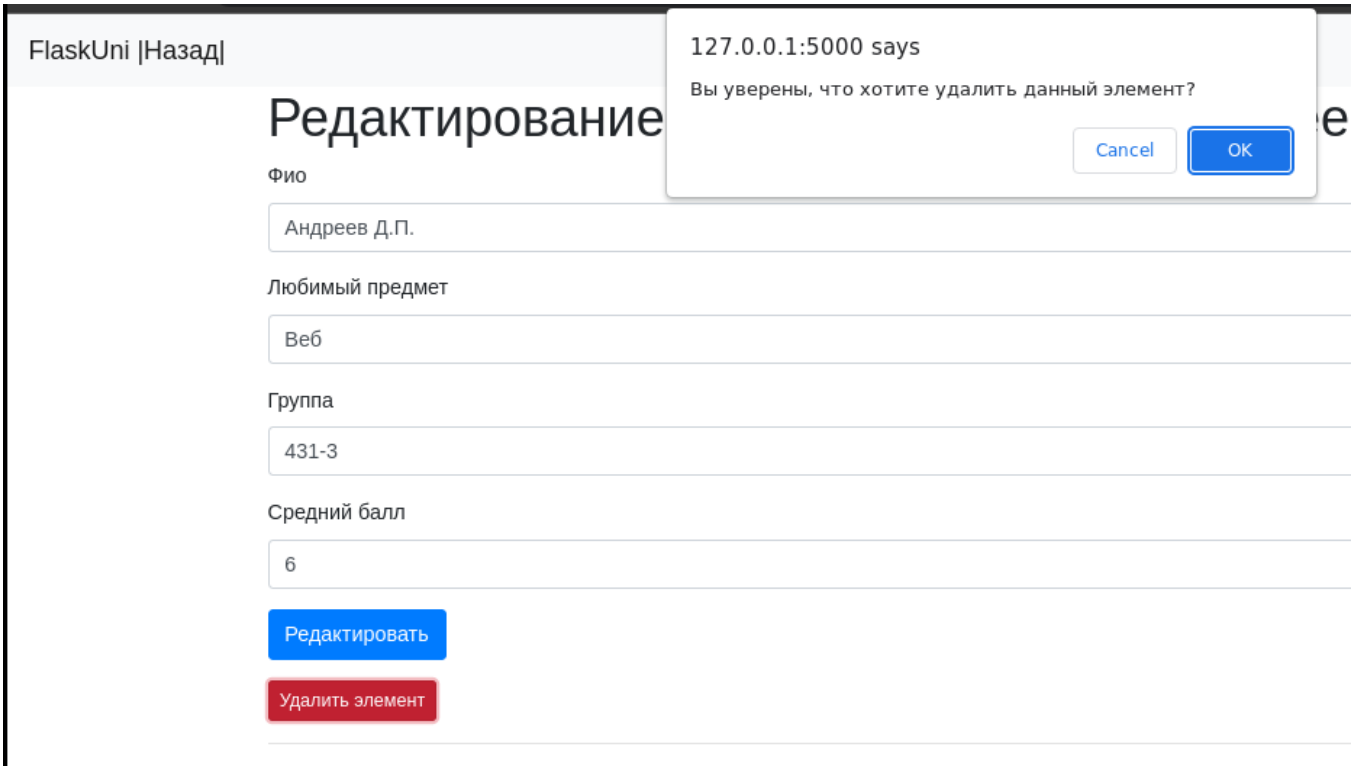


Рисунок 3.12 – удаление элемента

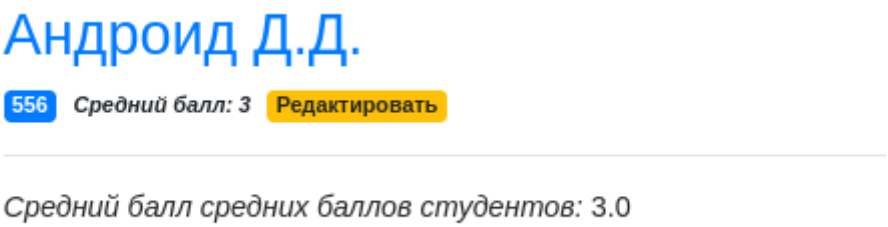


Рисунок 3.13 – результат удаления элемента

Также было решено сделать автоматическую документацию при помощи Swagger. Для каждой команды был присвоен свой личный комментарий, а также все используемые переменные (результат на рисунке 3.17).

Рисунок 3.14 – документация проекта при помощи Swagger

## Вывод

С помощью python-а можно решить различные задачи, связанные не только с математикой и с обработкой фото, но также и с созданием веб-приложений с интерфейсом. Также для веб-сервиса можно сделать автоматическую документацию.

## 4. Приложение

Листинги кодов решений:

App.py:

```
import sqlite3

from flask_swagger_ui import get_swaggerui_blueprint



from flask import Flask, render_template, request, url_for, flash, redirect
from flask import Flask, jsonify, Blueprint
from flasgger import Swagger
#from flask import g###
from werkzeug.exceptions import abort

SWAGGER_URL = '/docs'
API_URL = '/swagger'

swagger_ui_blueprint = get_swaggerui_blueprint(
  SWAGGER_URL,
  API_URL,
  config={
     'app_name': 'My App'
  }
)

def get_db_connection(): #Подключение к мини базе данных
    conn = sqlite3.connect('flask_blog/database.db') ##Путь до базы данных
    conn.row_factory = sqlite3.Row
    return conn

def get_post(post_id): #Открыть пост, если нету - дать нахуй по ебалу 404 ошибкой
    conn = get_db_connection()
    post = conn.execute('SELECT * FROM posts WHERE id = ?',
                        (post_id,)).fetchone()
    conn.close()
```

```
    if post is None:
        abort(404)
    return post

app = Flask(__name__)
swagger = Swagger(app) # Инициализация для нашего API сервиса документации Swagger

main = Blueprint("/", __name__, template_folder='templates',static_folder='static')

@main.route('/info/<about>/')
def info(about):
    """Возвращает информацию
 This is using docstrings for specifications.
 ---
 parameters:
  - name: about
    in: path
    type: string
    enum: ['all','version', 'author', 'year']
    required: true
    default: all
 definitions:
  About:
   type: string
 responses:
  200:
   description: A string
   schema:
    $ref: '#/definitions/About'
   examples:
    version: '1.0'
 """
    all_info = {
  'all': 'main_author 1.0 2020',
  'version': '1.0',
  'author': 'main_author',
```

```
  'year': '2020'
  }
    result = {about:all_info[about]}
    return jsonify(result)



@app.route('/<int:post_id>') #Открытие записей
def post(post_id):
    """чтение записи
 This is using docstrings for specifications.
 ---
 parameters:
  - name: get_post
    in: path
    type: integer
    enum: ['post_id']
    required: true
    default: all
 definitions:
  About:
   type: string
 responses:
  200:
   description: A string
   schema:
    $ref: '#/definitions/About'
   examples:
    version: '1.0'
 """
    post = get_post(post_id)
    return render_template('post.html', post=post)

@app.route('/swagger')
def create_swagger_spec():
   return json.dumps(get_apispec(app).to_dict())
```

```
@app.route('/create', methods=('GET', 'POST')) #Создание записей
def create():
        """Создание записи
 This is using docstrings for specifications.
 ---
 parameters:
  - name: get_post
    in: path
    type: integer
    enum: ['post_id']
    required: true
    default: all
 definitions:
  About:
   type: string
 responses:
  200:
   description: A string
   schema:
    $ref: '#/definitions/About'
   examples:
    version: '1.0'
 """
        if request.method == 'POST':
                title = request.form['title']
                performer = request.form['performer']
                style = request.form['style']
                duration = request.form['duration']

                if not title:
                        flash('Title is required!')
                else:
                        conn = get_db_connection()
                        conn.execute('INSERT INTO posts (title, performer, style, duration) VALUES
(?, ?, ?, ?)',
```

```
                    (title, performer, style, duration))
                        conn.commit()
                        conn.close()
                        return redirect(url_for('index'))
            return render_template('create.html')

@app.route('/') #Основная страница
def index():
    conn = get_db_connection()
    posts_var = conn.execute('SELECT * FROM posts').fetchall()
    min_var = conn.execute("SELECT MIN(duration) FROM posts").fetchone()[0]
    max_var = conn.execute("SELECT MAX(duration) FROM posts ").fetchone()[0]
    count_var = conn.execute("SELECT Count(title) FROM posts").fetchone()[0]
    summ_var = conn.execute("SELECT SUM(duration) FROM posts").fetchone()[0]
    conn.close()

    return render_template('index.html', posts=posts_var, count_var=count_var, min_var=min_var,
max_var=max_var, summ_var=summ_var)

@app.route('/sort') #Сортировка по возрастанию среднего балла
def sort():
        """сортировка по возрастанию среднего балла
 This is using docstrings for specifications.
 ---
 parameters:
  - name: get_post
    in: path
    type: integer
    enum: ['post_id']
    required: true
    default: all
 definitions:
  About:
   type: string
 responses:
  200:
```

```
  description: A string
  schema:
   $ref: '#/definitions/About'
  examples:
   version: '1.0'
"""
        conn = get_db_connection()
        posts_var = conn.execute('SELECT * FROM posts ORDER BY duration').fetchall()
        min_var = conn.execute("SELECT MIN(duration) FROM posts").fetchone()[0]
        max_var = conn.execute("SELECT MAX(duration) FROM posts ").fetchone()[0]
        count_var = conn.execute("SELECT Count(title) FROM posts").fetchone()[0]
        summ_var = conn.execute("SELECT SUM(duration) FROM posts").fetchone()[0]
        conn.close()

        return render_template('index.html', posts=posts_var, count_var=count_var,
min_var=min_var, max_var=max_var, summ_var=summ_var)

@app.route('/sort2') #Сортировка по убыванию среднего балла
def sort2():
        """сортировка по убыванию среднего балла
 This is using docstrings for specifications.
 ---
 parameters:
  - name: get_post
    in: path
    type: integer
    enum: ['post_id']
    required: true
    default: all
 definitions:
  About:
   type: string
 responses:
  200:
   description: A string
   schema:
```

```
    $ref: '#/definitions/About'
   examples:
    version: '1.0'
  """
        conn = get_db_connection()
        posts_var = conn.execute('SELECT * FROM posts ORDER BY duration DESC').fetchall()
        min_var = conn.execute("SELECT MIN(duration) FROM posts").fetchone()[0]
        max_var = conn.execute("SELECT MAX(duration) FROM posts ").fetchone()[0]
        count_var = conn.execute("SELECT Count(title) FROM posts").fetchone()[0]
        summ_var = conn.execute("SELECT SUM(duration) FROM posts").fetchone()[0]
        conn.close()
        return render_template('index.html', posts=posts_var, count_var=count_var,
min_var=min_var, max_var=max_var, summ_var=summ_var)

@app.route('/sort3') #Сортировка по возрастанию Фио
def sort3():
    """сортировка по возрастанию Фио
 This is using docstrings for specifications.
 ---
 parameters:
  - name: get_post
    in: path
    type: integer
    enum: ['post_id']
    required: true
    default: all
 definitions:
  About:
   type: string
 responses:
  200:
   description: A string
   schema:
    $ref: '#/definitions/About'
   examples:
    version: '1.0'
```

```
    """
        conn = get_db_connection()
        posts_var = conn.execute('SELECT * FROM posts ORDER BY title').fetchall()
        min_var = conn.execute("SELECT MIN(duration) FROM posts").fetchone()[0]
        max_var = conn.execute("SELECT MAX(duration) FROM posts ").fetchone()[0]
        count_var = conn.execute("SELECT Count(title) FROM posts").fetchone()[0]
        summ_var = conn.execute("SELECT SUM(duration) FROM posts").fetchone()[0]
        conn.close()

        return render_template('index.html', posts=posts_var, count_var=count_var, min_var=min_var,
max_var=max_var, summ_var=summ_var)

    @app.route('/sort4') #Сортировка по убыванию ФИО
    def sort4():
        """сортировка по убыванию Фио
    This is using docstrings for specifications.
    ---
    parameters:
     - name: get_post
       in: path
       type: integer
       enum: ['post_id']
       required: true
       default: all
    definitions:
     About:
      type: string
    responses:
     200:
      description: A string
      schema:
       $ref: '#/definitions/About'
      examples:
       version: '1.0'
    """
        conn = get_db_connection()
```

```
    posts_var = conn.execute('SELECT * FROM posts ORDER BY title DESC').fetchall()
    min_var = conn.execute("SELECT MIN(duration) FROM posts").fetchone()[0]
    max_var = conn.execute("SELECT MAX(duration) FROM posts ").fetchone()[0]
    count_var = conn.execute("SELECT Count(title) FROM posts").fetchone()[0]
    summ_var = conn.execute("SELECT SUM(duration) FROM posts").fetchone()[0]
    conn.close()

    return render_template('index.html', posts=posts_var, count_var=count_var, min_var=min_var,
max_var=max_var, summ_var=summ_var)

@app.route('/sort5') #Сортировка по возрастанию группы
def sort5():
    """сортировка по возрастанию группы
 This is using docstrings for specifications.
 ---
 parameters:
  - name: get_post
    in: path
    type: integer
    enum: ['post_id']
    required: true
    default: all
 definitions:
  About:
   type: string
 responses:
  200:
   description: A string
   schema:
    $ref: '#/definitions/About'
   examples:
    version: '1.0'
 """
    conn = get_db_connection()
    posts_var = conn.execute('SELECT * FROM posts ORDER BY style').fetchall()
    min_var = conn.execute("SELECT MIN(duration) FROM posts").fetchone()[0]
```

```
    max_var = conn.execute("SELECT MAX(duration) FROM posts ").fetchone()[0]
    count_var = conn.execute("SELECT Count(title) FROM posts").fetchone()[0]
    summ_var = conn.execute("SELECT SUM(duration) FROM posts").fetchone()[0]
    conn.close()

    return render_template('index.html', posts=posts_var, count_var=count_var, min_var=min_var, max_var=max_var, summ_var=summ_var)

@app.route('/sort6') #Сортировка по убыванию группы
def sort6():
    """сортировка по убыванию группы
 This is using docstrings for specifications.
 ---
 parameters:
  - name: get_post
    in: path
    type: integer
    enum: ['post_id']
    required: true
    default: all
 definitions:
  About:
   type: string
 responses:
  200:
   description: A string
   schema:
    $ref: '#/definitions/About'
   examples:
    version: '1.0'
 """
    conn = get_db_connection()
    posts_var = conn.execute('SELECT * FROM posts ORDER BY style DESC').fetchall()
    min_var = conn.execute("SELECT MIN(duration) FROM posts").fetchone()[0]
    max_var = conn.execute("SELECT MAX(duration) FROM posts ").fetchone()[0]
    count_var = conn.execute("SELECT Count(title) FROM posts").fetchone()[0]
```

```
    summ_var = conn.execute("SELECT SUM(duration) FROM posts").fetchone()[0]
    conn.close()

    return render_template('index.html', posts=posts_var, count_var=count_var, min_var=min_var, max_var=max_var, summ_var=summ_var)

@app.route('/sort7') #Сортировка по возрастанию любимого предмета
def sort7():
    """сортировка по возрастанию фамилии врача
 This is using docstrings for specifications.
 ---
 parameters:
  - name: get_post
    in: path
    type: integer
    enum: ['post_id']
    required: true
    default: all
 definitions:
  About:
   type: string
 responses:
  200:
   description: A string
   schema:
    $ref: '#/definitions/About'
   examples:
    version: '1.0'
 """
    conn = get_db_connection()
    posts_var = conn.execute('SELECT * FROM posts ORDER BY performer').fetchall()
    min_var = conn.execute("SELECT MIN(duration) FROM posts").fetchone()[0]
    max_var = conn.execute("SELECT MAX(duration) FROM posts ").fetchone()[0]
    count_var = conn.execute("SELECT Count(title) FROM posts").fetchone()[0]
    summ_var = conn.execute("SELECT SUM(duration) FROM posts").fetchone()[0]
    conn.close()
```

```
    return render_template('index.html', posts=posts_var, count_var=count_var, min_var=min_var, max_var=max_var, summ_var=summ_var)

@app.route('/sort8') #Сортировка по убыванию любимого предмета
def sort8():
    """Сортировка по убыванию фамилии врача
 This is using docstrings for specifications.
 ---
 parameters:
  - name: get_post
    in: path
    type: integer
    enum: ['post_id']
    required: true
    default: all
 definitions:
  About:
   type: string
 responses:
  200:
   description: A string
   schema:
    $ref: '#/definitions/About'
   examples:
    version: '1.0'
 """
    conn = get_db_connection()
    posts_var = conn.execute('SELECT * FROM posts ORDER BY performer DESC').fetchall()
    min_var = conn.execute("SELECT MIN(duration) FROM posts").fetchone()[0]
    max_var = conn.execute("SELECT MAX(duration) FROM posts ").fetchone()[0]
    count_var = conn.execute("SELECT Count(title) FROM posts").fetchone()[0]
    summ_var = conn.execute("SELECT SUM(duration) FROM posts").fetchone()[0]
    conn.close()
```

```
    return render_template('index.html', posts=posts_var, count_var=count_var, min_var=min_var, max_var=max_var, summ_var=summ_var)


@app.route('/<int:id>/edit', methods=('GET', 'POST')) #Редактирование полей
def edit(id):
    """Редатор записи
 This is using docstrings for specifications.
 ---
 parameters:
  - name: get_post
    in: path
    type: integer
    enum: ['post_id']
    required: true
    default: all
 definitions:
  About:
   type: string
 responses:
  200:
   description: A string
   schema:
    $ref: '#/definitions/About'
   examples:
    version: '1.0'
 """
    post = get_post(id)

    if request.method == 'POST':
        title = request.form['title']
        performer = request.form['performer']
        style = request.form['style']
        duration = request.form['duration']
```

```
        if not title:
            flash('Title is required!')
        else:
            conn = get_db_connection()
            conn.execute('UPDATE posts SET title = ?, performer = ?, style = ?, duration = ?'
                         ' WHERE id = ?',
                         (title, performer, style, duration, id))
            conn.commit()
            conn.close()
            return redirect(url_for('index'))

    return render_template('edit.html', post=post)



@app.route('/<int:id>/delete', methods=('POST','GET')) #Удаление записей
def delete(id):
    post = get_post(id)
    conn = get_db_connection()
    conn.execute('DELETE FROM posts WHERE id = ?', (id))
    conn.commit()
    conn.close()
    flash('"{}" was successfully deleted!'.format(post['title']))
    return redirect(url_for('index'))
"""Удаление записи
 This is using docstrings for specifications.
 ---
 parameters:
  - name: get_post
    in: path
    type: integer
    enum: ['post_id']
    required: true
    default: all
 definitions:
  About:
   type: string
```

```
 responses:
  200:
   description: A string
   schema:
    $ref: '#/definitions/About'
   examples:
    version: '1.0'
 """

app.register_blueprint(main,url_prefix='/')
app.run(debug=True)
```